

Instituto Tecnológico y de Estudios Superiores de Monterrey

Campus Monterrey

Escuela de Ingeniería y Ciencias



Proyecto UAVs

Materia:

TE3002B Implementación de robótica inteligente (Gpo. 102)

Estudiante

Roberto Raymundo Gómez Vargas A01285451

Docente:

PhD. Herman Castañeda Cuevas

Monterrey, Nuevo León, México, 2025

Resumen

En este proyecto de percepción visual y control, desarrollé un sistema de seguimiento facial autónomo utilizando un dron DJI Tello controlado desde una computadora mediante Python. Empleé la biblioteca *face_recognition* para detectar un rostro previamente registrado, e implementé un sistema de control reactivo que ajusta el movimiento del dron en los ejes vertical, rotacional y frontal para mantener el rostro centrado y a una distancia constante. Para mejorar la fluidez de la cámara, se incorporó multithreading, lo que permitió realizar el procesamiento de imagen y el control en tiempo real. Además, se integró lógica para evitar que el dron excediera una altura límite. Este trabajo representa un ejemplo práctico de cómo se puede combinar visión artificial con el control de un vehículo aéreo no tripulado (UAV) para tareas de seguimiento inteligente.

Palabras clave: Visión por computadora, Dron, Seguimiento facial, UAV, Control reactivo, Python, OpenCV, Percepción visual

Índice general

1. Acerca de la situación problema	3
1.1. Introducción	3
1.2. Descripción del Proyecto	4
2. Desarrollo	5
2.1. Herramientas y Tecnologías Utilizadas	5
2.1.1. Lenguaje y entorno	5
2.1.2. Bibliotecas utilizadas	5
2.1.3. Lógica de control	6
2.2. Código	6
2.3. Explicación del código	9
2.3.1. Carga del rostro de referencia	9
2.3.2. Conexión con el dron	10
2.3.3. Multithreading para procesamiento fluido	10
2.3.4. Despegue automático	10
2.3.5. Bucle principal de detección y control	10
2.3.6. Interfaz visual	10
2.3.7. Aterrizaje y cierre del sistema	11
3. Resultados	12
3.1. Dron POV	12
3.2. External POV	12
3.3. Conclusión	12
Bibliografía	13

Índice de figuras

1.1. Diagrama	4
-------------------------	---

Capítulo 1

Acerca de la situación problema

1.1. Introducción

En la última década, los vehículos aéreos no tripulados (UAVs), y en particular los cuadricópteros, han cobrado gran relevancia en diversas áreas como vigilancia, entrega de paquetes, agricultura de precisión, monitoreo ambiental y aplicaciones educativas. Gracias a su capacidad de despegue y aterrizaje vertical (VTOL), así como a su maniobrabilidad en espacios reducidos, los cuadricópteros se han convertido en una plataforma ideal para la investigación y el desarrollo de sistemas autónomos.

Estos dispositivos combinan diversas disciplinas de la ingeniería como el control automático, la electrónica, la programación y la percepción visual. Su estabilidad en vuelo se logra mediante algoritmos de control que ajustan continuamente las velocidades de los motores en respuesta a los cambios del entorno. Además, el avance en tecnologías de visión por computadora ha permitido incorporar capacidades de navegación y seguimiento inteligentes, haciendo posible que estos sistemas reaccionen de manera autónoma ante estímulos visuales.

Dentro de un contexto educativo, trabajar con cuadricópteros como el DJI Tello permite a los estudiantes experimentar de forma práctica conceptos de robótica, programación y control, integrando herramientas modernas como Python, OpenCV y bibliotecas de reconocimiento facial. Este tipo de proyectos no solo refuerzan los conocimientos teóricos adquiridos en clase, sino que también desarrollan habilidades de resolución de problemas en tiempo real y diseño de sistemas complejos.

1.2. Descripción del Proyecto

Este proyecto consistió en desarrollar un sistema que permitiera a un dron DJI Tello seguir automáticamente mi rostro mientras volaba. La idea principal fue lograr que el dron pudiera reconocerme visualmente, y que ajustara su posición de manera autónoma para mantenerse centrado en mi cara y a una distancia adecuada.

Para esto, cargué previamente una imagen de referencia con mi rostro, y a partir de ahí configuré el sistema para que lo identificara en tiempo real mientras el dron transmitía video. El objetivo era que el dron se moviera hacia arriba, abajo, girara, o avanzara y retrocediera según la posición de mi cara en la imagen.

Además, añadí una función para que el dron dejara de moverse si dejaba de verme (hover), y establecí un límite de altura para evitar que subiera demasiado. También me aseguré de que la cámara funcionara de manera fluida, ya que era importante que el sistema reaccionara rápido a los movimientos.

Este proyecto me permitió aplicar de manera práctica conceptos de percepción visual, automatización y control, todo enfocado a lograr que un dron se comportara de manera inteligente ante estímulos visuales.

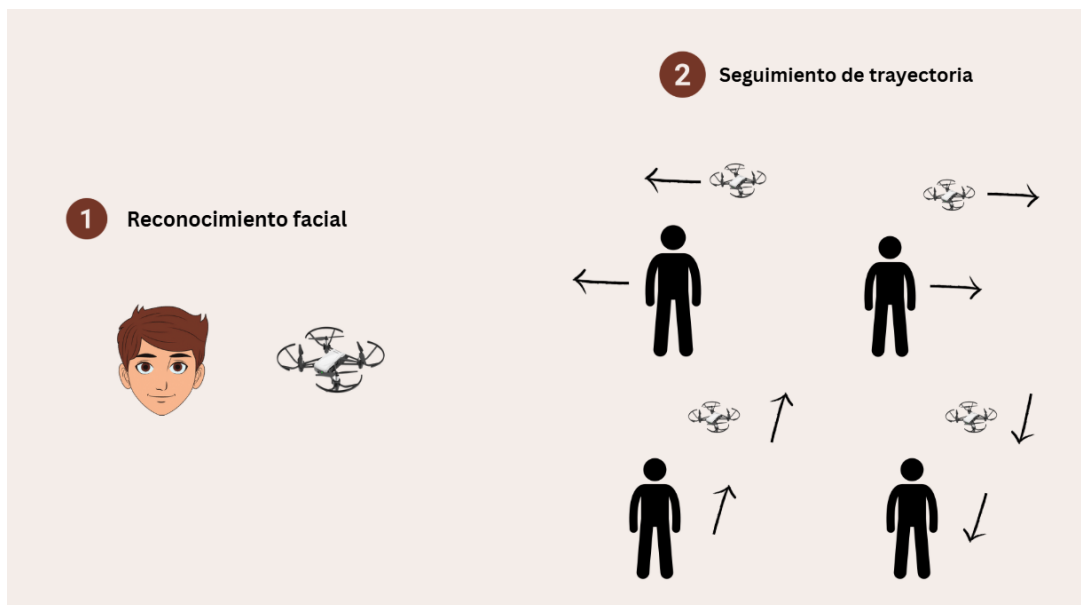


Figura 1.1: Diagrama

Capítulo 2

Desarrollo

2.1. Herramientas y Tecnologías Utilizadas

Para implementar el sistema de seguimiento facial con el dron DJI Tello, utilicé principalmente el lenguaje de programación Python junto con varias bibliotecas especializadas en visión por computadora, control y comunicación con el dron. A continuación, se describen las herramientas utilizadas:

2.1.1. Lenguaje y entorno

- **Python 3.10:** Fue el lenguaje principal del proyecto por su facilidad para el procesamiento de imágenes y por la disponibilidad de bibliotecas que facilitan el control del dron y la visión artificial.
- **Visual Studio Code:** Utilizado como entorno de desarrollo por su integración con Python y su soporte para proyectos organizados.

2.1.2. Bibliotecas utilizadas

- **djitellopy:** Esta biblioteca permite controlar el dron DJI Tello mediante comandos en Python. Se utilizó para conectar el dron, obtener el video en tiempo real, enviar comandos de movimiento, y monitorear su batería y altura.
- **face_recognition:** Utilizada para detectar y reconocer el rostro del usuario a partir de una imagen de referencia. Esta biblioteca se basa en modelos preentrenados y permite comparar rostros con un parámetro de tolerancia para determinar si coinciden.
- **OpenCV (cv2):** Usada para procesar las imágenes de la cámara del dron. Con OpenCV se pudo redimensionar el video, convertirlo a RGB, dibujar cuadros y textos en pantalla, y mostrar el flujo en tiempo real.

- **threading:** Fue necesario usar hilos (threads) para separar el procesamiento de la cámara del resto de la lógica del sistema, permitiendo así una experiencia más fluida sin que el control del dron se viera afectado por la latencia del video.

2.1.3. Lógica de control

- Se cargó una imagen de referencia con mi rostro, y se extrajo su codificación facial.
- El dron despegaba y comenzaba a transmitir video en tiempo real.
- Cada ciertos cuadros, se analizaba la imagen para buscar un rostro y comparar si coincidía con el previamente guardado.
- Si se detectaba el rostro correcto:
 - El sistema calculaba la posición del rostro en la imagen para saber si estaba centrado.
 - Con base en la posición del rostro, se ajustaban los ejes del dron: rotación (yaw), altura (up/down) y distancia (adelante/atrás) para mantener una distancia visual constante.
 - Se estableció una lógica para evitar que el dron superara los 190 cm de altura.
 - También se definió una zona de tolerancia para no hacer ajustes innecesarios si el rostro ya estaba dentro de un rango aceptable.
- Si no se detectaba el rostro, el dron se detenía (hover).

2.2. Código

```
import cv2
import face_recognition
import threading
from djitellopy import Tello
import time

# Cargamos la imagen de referencia del rostro (en este caso, el mío) y obtenemos su
↳ codificación
known_image = face_recognition.load_image_file("known_face.jpg")
known_encoding = face_recognition.face_encodings(known_image)[0]

# Conectamos con el dron Tello, activamos el stream de video y esperamos un momento para
↳ estabilizar la imagen
tello = Tello()
tello.connect()
print(f" Battery: {tello.get_battery()}%")
tello.streamon()
frame_reader = tello.get_frame_read()
time.sleep(2)
```



```
# Configuración de multithreading para que el video se procese en paralelo y no haya tanto lag
frame_lock = threading.Lock()
latest_frame = None
running = True

# Esta función corre en segundo plano y actualiza continuamente el último frame capturado
def camera_loop():
    global latest_frame, running
    while running:
        frame = frame_reader.frame
        if frame is not None:
            with frame_lock:
                latest_frame = frame.copy()

# Iniciamos el hilo del video
threading.Thread(target=camera_loop, daemon=True).start()

# El dron despegua y sube a una altura inicial segura
print(" Preparing for takeoff...")
try:
    tello.takeoff()
    time.sleep(1)
    tello.move_up(80)
except Exception as e:
    print(f" Takeoff failed: {e}")
    tello.end()
    exit()

# Variables de control para el bucle principal
print(" Face tracking with distance control. Press 'q' to quit.")
frame_count = 0
recognized_faces = []
DESIRED_FACE_WIDTH = 170 # Tamaño ideal del rostro en píxeles (aprox. 1 metro de distancia)
TOLERANCE = 15          # Tolerancia para no moverse todo el tiempo

# Bucle principal de procesamiento y control
while True:
    # Tomamos el frame más reciente protegido por el lock del hilo
    with frame_lock:
        frame = latest_frame.copy() if latest_frame is not None else None
    if frame is None:
        continue

    frame_count += 1
    frame_h, frame_w, _ = frame.shape
    face_width = 0
    face_found = False

    # Procesamos cada 10 frames para reducir la carga de CPU
    if frame_count % 10 == 0:
        # Redimensionamos y convertimos el frame a RGB para usarlo con face_recognition
```

```

small_frame = cv2.resize(frame, (0, 0), fx=0.25, fy=0.25)
rgb_small_frame = cv2.cvtColor(small_frame, cv2.COLOR_BGR2RGB)

# Detectamos rostros y obtenemos sus codificaciones
face_locations = face_recognition.face_locations(rgb_small_frame)
face_encodings = face_recognition.face_encodings(rgb_small_frame, face_locations)
recognized_faces = []

# Comparamos cada rostro detectado con el de referencia
for face_encoding, location in zip(face_encodings, face_locations):
    match = face_recognition.compare_faces([known_encoding], face_encoding,
    ↪ tolerance=0.8)[0]
    label = "Raymundo" if match else "Not Raymundo"
    color = (0, 255, 0) if match else (0, 0, 255)
    recognized_faces.append((location, label, color))

if label == "Raymundo":
    face_found = True
    # Reescalamos coordenadas porque originalmente redujimos el tamaño de la imagen
    top, right, bottom, left = location
    top *= 4; right *= 4; bottom *= 4; left *= 4
    cx = (left + right) // 2
    cy = (top + bottom) // 2
    face_width = right - left # Cuántos píxeles de ancho ocupa el rostro

    # Calculamos los errores respecto al centro de la imagen
    error_x = cx - (frame_w // 2)
    error_y = cy - (frame_h // 2)
    yaw_speed = int(error_x * 0.1)
    updown_speed = int(error_y * -0.1)

    # Control frontal: acercarse o alejarse si la cara es muy grande o muy pequeña
    if face_width < (DESIRED_FACE_WIDTH - TOLERANCE):
        fb_speed = 15
    elif face_width > (DESIRED_FACE_WIDTH + TOLERANCE):
        fb_speed = -15
    else:
        fb_speed = 0

    # Limito las velocidades para que el dron no haga movimientos bruscos
    yaw_speed = max(-30, min(30, yaw_speed))
    updown_speed = max(-20, min(20, updown_speed))
    fb_speed = max(-20, min(20, fb_speed))

    # Seguridad para no pasar de 190 cm de altura
    if tello.get_height() >= 190 and updown_speed > 0:
        updown_speed = 0

    # Enviamos los comandos de movimiento al dron
    tello.send_rc_control(0, fb_speed, updown_speed, yaw_speed)

# Si no encuentro mi cara, detengo el movimiento del dron

```

```
if not face_found:
    tello.send_rc_control(0, 0, 0, 0)

# Dibujo los rostros detectados
for (top, right, bottom, left), label, color in recognized_faces:
    top -= 4; right -= 4; bottom -= 4; left -= 4
    cv2.rectangle(frame, (left, top), (right, bottom), color, 2)
    cv2.putText(frame, label, (left, top - 10), cv2.FONT_HERSHEY_SIMPLEX, 0.7, color, 2)

# Muestro en pantalla cuántos píxeles ocupa el rostro
cv2.putText(frame, f"Face Width: {face_width}px", (10, 60),
            cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255, 255, 0), 2)

# Mostramos la vista de la cámara del dron
cv2.imshow("Drone View", frame)
if cv2.waitKey(1) & 0xFF == ord('q'):
    print(" Landing...")
    break

# Apagado seguro del sistema
tello.send_rc_control(0, 0, 0, 0)
tello.land()
running = False
tello.streamoff()
tello.end()
cv2.destroyAllWindows()
print(" Done.")
```

2.3. Explicación del código

El código desarrollado tiene como objetivo implementar un sistema de seguimiento facial con control reactivo en un dron DJI Tello, manteniendo al rostro centrado en la imagen, a una altura constante y a una distancia determinada. A continuación se explican los principales bloques del código:

2.3.1. Carga del rostro de referencia

Primero se utiliza la biblioteca `face_recognition` para cargar una imagen del rostro de referencia. Esta imagen se codifica en un vector de características para poder comparar rostros detectados en tiempo real.

2.3.2. Conexión con el dron

Se establece conexión con el dron DJI Tello utilizando la biblioteca `djitellopy`. También se inicia la transmisión de video y se obtiene el objeto que permite leer los cuadros capturados por la cámara.

2.3.3. Multithreading para procesamiento fluido

Para evitar retrasos al procesar los cuadros de video, se emplea `multithreading`. Se crea un hilo dedicado a actualizar constantemente el último frame disponible, lo que permite que el procesamiento de imagen y el control se ejecuten sin interferencias.

2.3.4. Despegue automático

Una vez conectado, el dron despegue automáticamente y sube a una altura inicial segura. Este paso también está protegido por un bloque `try-except` para manejar cualquier posible error de despegue.

2.3.5. Bucle principal de detección y control

Dentro del bucle principal se hace lo siguiente:

- Se procesa la imagen cada 10 cuadros para reducir la carga computacional.
- Se detectan rostros en la imagen y se comparan con el rostro previamente cargado.
- Si se detecta el rostro correcto, se calcula su posición en la imagen y su tamaño en píxeles.
- Se aplican controles en tres ejes:
 - **Eje vertical:** El dron sube o baja si el rostro está descentrado verticalmente.
 - **Eje rotacional (yaw):** El dron rota si el rostro está descentrado horizontalmente.
 - **Eje frontal (adelante/atrás):** Se calcula el ancho del rostro. Si el tamaño es menor al deseado, el dron avanza; si es mayor, retrocede.
- Se limita la velocidad de cada eje para evitar movimientos bruscos.
- También se incluye una condición de seguridad para evitar que el dron suba más de 190 cm.

2.3.6. Interfaz visual

Se dibujan recuadros alrededor de los rostros detectados y se muestra en pantalla el ancho del rostro principal en píxeles. Esto permite verificar visualmente el funcionamiento del sistema.

2.3.7. Aterrizaje y cierre del sistema

Cuando el usuario presiona la tecla **q**, el dron aterriza de forma segura, se detienen los hilos, se apaga la cámara y se cierra la ventana del video.

Capítulo 3

Resultados

3.1. Dron POV

→ [Video demostrativo](#)

3.2. External POV

→ [Video demostrativo](#)

3.3. Conclusión

Este proyecto me permitió integrar habilidades de percepción visual, programación y control en un sistema real con un dron. A través de la implementación del seguimiento facial en el DJI Tello, logré que el dron reaccionara de forma autónoma a la posición de mi rostro, manteniéndome centrado en el encuadre, a una altura adecuada y a una distancia constante.

Más allá de los resultados técnicos, este reto representó una excelente oportunidad para fortalecer mi capacidad de análisis, solución de problemas y aplicación de tecnologías modernas como `OpenCV`, `face_recognition`, y control reactivo en tiempo real. El uso de multithreading mejoró notablemente el desempeño del sistema, permitiendo que el procesamiento y la respuesta del dron fueran fluidos.

Considero que este proyecto refleja el tipo de integración entre software y hardware que se requiere en aplicaciones reales de robótica, visión por computadora y vehículos autónomos. Me voy satisfecho con lo aprendido y con la confianza de poder enfrentar proyectos más complejos en el futuro.

Bibliografía

(2024). Opencv: Open source computer vision library. OpenCV Official Website. <https://opencv.org>.

(2024). Tello drone — ryze robotics. Ryze Robotics Website. <https://www.ryzerobotics.com/tello>.

Foundation, P. S. (2024). threading — thread-based parallelism. Python Docs. <https://docs.python.org/3/library/threading.html>.

Geitgey, A. (2024). face_recognition python library. Documentation. <https://face-recognition.readthedocs.io/en/latest/>.